



THM

**CAMPUS
GIESSEN**

MNI

Mathematik, Naturwissenschaften
und Informatik

TECHNISCHE HOCHSCHULE MITTELHESSEN

Generative Jazz Musik

Projektdokumentation (Gruppe A)

Alexandra Schnakenberg

Fabian Manger

Peer Schliephacke

Grundlagen der Künstlichen Intelligenz

Prof. Dr.-Ing. Seyed Eghbal Ghobadi

Technische Hochschule Mittelhessen

Wintersemester 2022/2023

17. Juni 2026



Inhaltsverzeichnis

I	Einleitung	1
II	Verwandte Arbeiten	1
III	Daten	2
1	Sammeln von Trainingsdaten	2
2	Normalisieren	2
3	Prozessierung der Daten	3
IV	Architektur	4
1	Modell-Architektur	4
2	Training	5
3	Hyperparameter	5
4	MIDI-Generierung	6
V	Validierung	7
1	Statistische Analyse	7
A	Tonhöhe	7
B	Tondauer	8
C	Overfitting	8
2	Imitation Game	9
	Literatur	11

Abbildungsverzeichnis

1	Beispiel Lick	1
2	Schema: Datenprozessierung	4
3	Architektur des Modells	5
4	MIDI Generierung	6
5	Boxplot kodierte Tonhöhenverteilung	7
6	Barplot kodierte Tonhöhenverteilung	7
7	Boxplot kodierte Tondauer	8
8	Overfitting Line Chart	8
9	Computer Generiert Barplot	9
10	Bewertung Boxplot	9

I Einleitung

Das vorliegende Projekt bezieht sich auf die Verwendung von Neuronalen Netzen zur Generierung von kurzen musikalischen monophonen Phrasologien, die im Jazz-Bereich als *Licks* bekannt sind. Jazz umfasst eine Musikrichtung, die sich vor allem durch einen freien kreativen Umgang bzgl. melodischer Konstrukte und harmonischer Struktur ausdrückt. Infolgedessen lebt die Jazz-Musik von improvisatorischen Elementen, die einerseits auf bewährte Erfahrungen und andererseits auf gut durchdachte musiktheoretische Kenntnisse zurückgreift. Im Laufe der Jazz-Historie sind viele kurze musikalische Abschnitte entstanden, die für Improvisationen immer wieder genutzt werden. Derartige kurze markante Abschnitte können von anderen Musikern gelernt werden und infolge eigener Improvisationen wiederverwendet werden. Solche Abschnitte werden *Licks* genannt und können durchaus als musikalische Zitate beschrieben werden.

Solche Zitate sind an einen bestimmten harmonischen Verlauf gekoppelt (üblicherweise eine II-V-I Kadenz) und bedienen sich diverser musikalischer Skalen. Ein funktionierendes *Lick* sollte entsprechend der gegenwärtigen Harmonie des jeweiligen Taktes auf einer korrespondierenden Skala basieren. Entsprechend werden viele theoretische Kenntnisse vorausgesetzt, um *Licks* zu kreieren. Auf der anderen Seite können bestehende *Licks* genutzt werden, um bewährtes musikalisches Wissen zugunsten einer interessanten Improvisation zu verwenden (quasi ein Echtzeit-Zitieren während einer Improvisation).



Abbildung 1. Ein einfaches *Lick* über eine II-V-I Kadenz in Dur

Abbildung 1 zeigt exemplarisch ein grundlegendes *Lick*: Der erste Takt (dm^7) besteht aus ei-

ner dorisch skalierten Melodie, während sich der zweite Takt (G^7) einer mixolydischen ($b13$) Skala bedient. Der dritte und vierte Takt ($C^{\Delta 7}$) korrespondiert zum tonalen Zentrum und ist in C-ionisch geschrieben.

Diese Projektarbeit soll zeigen, dass Neuronale Netze die Fähigkeit besitzen, Kunst in Form von Jazz-Musik künstlich zu generieren. Diese Generierung von *Jazz-Licks* erfolgt somit ohne Kenntnisse über funktionsharmonischen oder skalentheoretische Konzepten der Musik, sondern durch bloßes Verallgemeinern von Mustern in der Trainingsdaten (Musik).

II Verwandte Arbeiten

Durch die vermehrte *KI*-Nutzung und -Forschung existieren zahlreiche ähnliche Projekte zur künstlichen Generierung von Kunst. Dabei lassen sich facettenreiche Implementierungen und Kunstgattungen unterscheiden. Beispielsweise existieren somit unterschiedlichste Modelle, die zur Generierung von Bildern, literarischen Texten oder Musik implementiert wurden. Jene Modelle unterscheiden sich wiederum in der Architektur oder dem Format der Eingabedaten und somit auch in den entsprechenden Subgattungen, wie etwa die Musikrichtung oder der Kunststil.

Die vorliegende Projektarbeit wurde maßgeblich von einigenden bestehenden Projekten beeinflusst, da die Thematisierung von generativen Netzen (und rekurrenter Netze (RNN)) in der Vorlesung nur peripher gestreift wurde und entsprechend keine Vorerfahrungen vorhanden waren.

Als erster Anhaltspunkt diente dabei der Artikel *Neural Networks for Music Generation* [1], welcher einen groben Überblick über infrage kommende Netzwerke gab, sowie Besonderheiten auf die zu achten sind. Letztlich entschieden wir uns für eine besondere Art von rekurrenten Netzen, den *Long-Short-Term-Memory Networks (LSTM)*. Obwohl LSTMs in dem genannten Ar-

tikel nicht die Bestleistung erreichten, entschieden wir uns dennoch für dieses Modell, da LSTMs vergleichsweise 'alt' sind und entsprechend weniger Vorkenntnisse benötigt werden und mehr Ressourcen zur Verfügung stehen.

Folglich gab es auch konkrete Implementierungen die uns halfen, den Aufbau von *KI*-Projekten und vor allem der Architektur von LSTMs zu verstehen. Jordan Birds [2] und Skuldurs [3] Repository vermittelten somit wertvolle Eindrücke für eine Implementierung und vor allem zur Prozessierung und Verarbeitung der Trainingsdaten im MIDI-Format.

Entsprechende weiterführende Recherche zeigte, dass die Verwendung des sogenannten *Attention-Mechanism* für die Arbeit mit Sequenzen sehr vielversprechend zu sein scheint. Da die Folge von Noten und der Notation des Rhythmus auch als eine Sequenz betrachtet werden kann, kombinierten wir die Idee unseres LSTMs mit einem *Attention-Mechanism*. Zum Verständnis des besagten Mechanismus wurde diverse Literatur genutzt ([4], [5], [6]). Eine konkrete Implementierung konnte in einem Git-Repository eingesehen werden [7].

Darüber hinaus wurde vermehrt Literatur genutzt um die nötigen Implementierungen umsetzen zu können. Dies beinhaltet den Umgang mit Daten, Einstieg in generative Modelle sowie das Aufstellen von RNN-Architekturen und *Attention* ([5], [8], [9], [10], [11]).

III Daten

Dieser Abschnitt beschreibt die Struktur des Datensatzes. Da der Datensatz eigenständig erzeugt wurde, wird in den folgenden Unterabschnitten beschrieben, wie das Erzeugen und Filtern der Trainingsdaten ausgesehen hat. Außerdem werden angewendete Techniken erwähnt, die genutzt wurden, um die Trainingsdaten in ein adäquates Format zu bringen.

1 Sammeln von Trainingsdaten

Da noch kein generatives Neuronales Netz zur Generierung von *Jazz-Licks* gefunden werden konnte, mussten die Trainingsdaten eigenhändig gesammelt, normalisiert und digitalisiert werden.

Das Sammeln der Trainingsdaten geschah mit *Lick*-Sammlungen von berühmten Jazz-Musikern (Pat Martino, John Coltraine, Jerry Bergonzi, Charlie Parker), Jazz-Literatur [12] oder *Lick*-Sammlungen die im Internet zur Verfügung stehen. Nach der Sichtung von Rohmaterial, wurden die einzelnen dreitaktigen Partituren mithilfe des Programms *Musescore* digitalisiert und in ein MIDI-Format exportiert. Das Digitalisieren konnte nicht automatisiert werden, weshalb jedes einzelne *Lick* abgetippt wurde. Weiterhin wurden die Trainingsdaten auch nicht vervielfältigt (Bspw. mit *Data Augmentation*), da kein Verfahren bekannt ist, wie die Ästhetik und die harmonische Struktur eines *Jazz-Licks* gleichzeitig erhalten werden können. Durch *Data Augmentation* Verfahren könnten die einzelnen melodischen Bestandteile zwar permutiert oder ähnlich kombinatorisch verändert werden, was jedoch nicht unbedingt zu gut klingenden *Licks* führen wird.

2 Normalisieren

Da *Jazz-Licks* eine große Varianz untereinander aufzeigen und durch die kleine Menge an verfügbaren Trainingsmaterial nur wenige Trainingsdaten resultierten, mussten einige Maßnahmen zur Verbesserung der Varianz gemacht werden.

Entsprechende Maßnahmen charakterisieren sich durch ein Transformieren der gesammelten *Licks*. Somit wurden alle *Licks* erstmal in die gleiche Tonart gebracht (Bb-Dur). Daran anknüpfend mussten einige *Licks* auch modifiziert werden, da sie entweder rhythmische Anomalien aufwiesen, welche den Datensatz verzerrt hätten oder aus zu vielen bzw. wenigen Takten be-

standen. Dieser Schritt konnte nur mit entsprechenden Domänenwissen durchgeführt werden, da beim Verändern der Trainingsdaten musiktheoretische Kenntnisse unabdingbar sind. Somit wurden rhythmische Anomalien geglättet, fehlende Takte mit entsprechenden Skalenmaterial hinzugefügt oder überschüssige Takte sowie Auftakte verkürzt. Resultierend gab es nach dem Modifizieren keine rhythmischen Auffälligkeiten, sowie eine gleiche Taktanzahl unter den *Licks*.

Da sich *Jazz-Licks* weiterhin in ihrem Tonmaterial unterscheiden können, wurde die Gesamtmenge an Trainingsdaten in zwei Kategorien aufgeteilt: *diatonisch* und *alteriert*. *Diatonische Licks* beschreiben dabei *Jazz-Licks* die weitestgehend nur aus Tonleitereigenem Material bestehen, *alterierte Licks* beinhalten darüber hinaus vermehrt Tonleiter-fremdes Material, welches sich eher mit Spannungen auf der Dominante ausdrücken lässt. Da die beiden Arten somit starke Unterschiede aufweisen, kann das bei niedrigerer Trainingsdatenzahl ein Verallgemeinern erschweren. Zugunsten einer erfolgreichen Verallgemeinerung wurden drei Modelle erstellt, die sich nur in den Trainingsdaten unterscheiden (*alteriert*, *diatonisch*, beides).

Da weder das Modifizieren noch das Filtern ohne musiktheoretische Kenntnisse machbar war, konnte weiterhin keine Automatisierung stattfinden, weshalb das Filtern und Modifizieren manuell in Musecore durchgeführt wurde.

3 Prozessierung der Daten

Nachdem die Trainingsdaten mithilfe von Musecore und entsprechendem Domänenwissen normalisiert wurden, mussten die Datenpunkte in ein Format gebracht werden, mit dem das Netzwerk trainieren kann.

Dafür wurden viele Funktionen eigenhändig implementiert oder von den zuvor genannten Git-Repositorys inspiriert, um das Einlesen und Be-

arbeiten der Daten zu ermöglichen. Viele dieser Funktionen basieren auf dem Pythonmodul *music21*, mit dem die MIDI-Dateien eingelesen werden konnten. Weiterhin wurden Funktionen implementiert, um aus jeder Datei eine Sequenz an Tonhöhen (*note*) und den entsprechenden rhythmischen Dauern (*duration*) zu extrahieren. Diese Kenngrößen wurden jeweils in einer Liste gespeichert und mit wiederholenden Symbolen (*Tokens*) getrennt. Die einzigartigen Tonhöhen, sowie Dauer der Rhythmen wurden weiterhin in Zahlen kodiert, da ein Neuronalesnetz nur mit numerischen Vektoren bzw. Matrizen trainiert werden kann. Da die Tonhöhen/Dauern kodiert werden, stellen sie kategorische bzw. ordinalskalierte Daten dar.

Der Vektor mit den Tonhöhen/Dauern, sowie die Hashmap zum Kodieren konnten genutzt werden, um die Daten weiterhin zu verarbeiten. Dafür wurde über ein *Sliding Window*, mit einer festgelegten Länge, iteriert. Dadurch wurde aus jedem *Trainings-Lick* eine Subsequenz der eigentlich Tonhöhen/Dauer-Sequenz entnommen werden und wiederum in einer eigenen Datenstruktur in kodierter Form gespeichert werden. Entsprechend resultierten dabei ein *Input-Vector* für Tonhöhe/Dauer, der die kodierten Subsequenzen enthält, sowie ein *Output-Vector* der mit der kodierten Anfangsnote der Subsequenz mithilfe des gleichen Indexes zu der eigentlich Subsequenz korrespondiert. Die *Input-Vectors* wurden weiterhin in eine Matrix überführt mit der Dimension (Länge des *Input – Vectors* × festgelegte Länge der Vorhersage). Die *Output-Vectors* wurden ebenfalls in eine Matrix überführt. Dabei wurde jede kodierte Anfangsnote mittels *One-Hot-Coding* erneut kodiert. Die Anzahl an Klassen entspricht dabei der Anzahl an eindeutigen Tonhöhen/Dauern. Die *Input/Output-Vectors* konnten nun verwendet werden, um entsprechende Gewichte zu erzeugen. Die *One-Hot-Coded* Tonhöhen/Dauern wurden ferner verwendet, um dem Neuronalen Netz aufzuzeigen, welche No-

tensequenz (*Input*) nach einer bestimmten Note (*Output*) folgen soll. Aus dieser Information kann das Netzwerk schließlich neue Sequenzen erzeugen, indem eine Wahrscheinlichkeitsverteilung simuliert wird, aus welcher hervorgeht, welche Note/Dauer nach einer vorhergehenden Note/Dauer am wahrscheinlichsten ist.

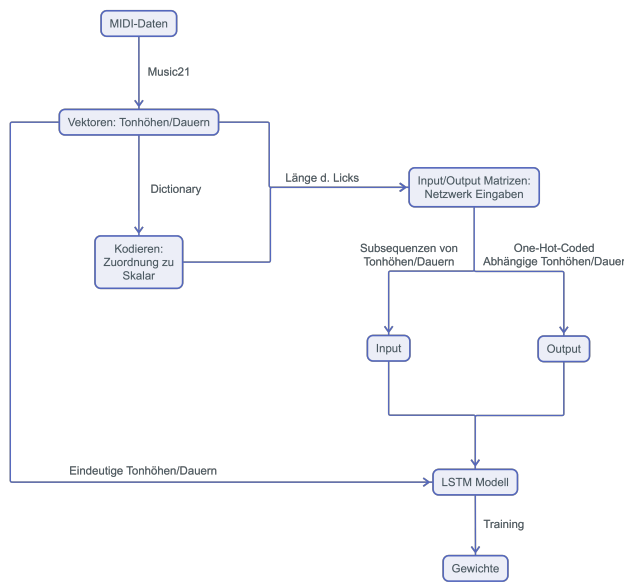


Abbildung 2. Schematische Darstellung der Generierung von Trainingsdaten aus gewöhnlichen MIDI Daten

Abbildung 2 fasst weitestgehend die Datenverarbeitung nach der manuellen Datensammlung und -normalisierung zusammen. Dabei werden die MIDI Daten zunächst mittels Funktionalitäten von *music21* in Python geladen und als zwei Vektoren pro Sample dargestellt: Ein Vektor für die Tonhöhe und ein Vektor für die Tondauer. Aus der Anzahl an Tonhöhen bzw dauern kann dann ein LSTM Modell erstellt werden. Die Vektoren werden weiterhin in Zahlen überführt, die wiederum weiterverarbeitet werden. Dabei wird jeder Ton in einer Sequenz an Tönen oder Tondauern mit den nachfolgenden Tonhöhen/Dauer-Subsequenzen assoziiert. Dabei stellt die Subsequenz den Input dar, während der Output die einzelne Tonhöhe/Dauer darstellt (*One-Hot-Coded*).

IV Architektur

1 Modell-Architektur

Das Modell nimmt zwei Eingaben entgegen, die Tonhöhen (*note_input*) und die Tondauer bzw. den Rhythmus (*dur_input*) (siehe Abbildung 3). Die Eingabevektoren werden dann in der *Embedding*-Schicht jeweils wie in Abschnitt Prozessierung der Daten kodiert und in einer *Concatenate*-Schicht zu einem Vektor zusammengeführt. Dieser wird dann in zwei LSTM-Schichten (*First LSTM*, *Second LSTM*) mit je 256 Neuronen verarbeitet. Diese leiten alle *hidden states* an die jeweils nächste Schicht. Um ein *Overfitting* zu vermeiden, wird auf die Ausgabe der zweiten LSTM-Schicht ein *Dropout* von 0,3 angewendet. Darauf folgt der Teil des Netzes, welcher den *Attention*-Mechanismus implementiert. Im Allgemeinen dient *Attention* dazu, die relevanten Positionen in der gesamten Sequenz für die nächste Vorhersage zu ermitteln. Hierfür wird ein weiterer Gewichtungsvektor berechnet, der die Relevanz jeder Position bewertet. In unserem Modell läuft dies wie folgt ab: Zunächst werden alle *hidden states* durch eine *Dense*-Schicht (*Adapt_layer*) mit tanh als Aktivierungsfunktion geleitet und diese Ausgabe in einer *Reshape*-Schicht (*Rm_1_vec*) in einen Vektor mit der Länge der Eingabesequenz umgeformt. Als nächstes wird die *Softmax*-Aktivierungsfunktion (*Softmax_Act*) angewandt. Der daraus resultierende Gewichtungsvektor wird 256 mal kopiert (*repeat_vector*), sodass sich eine Matrix mit so vielen Zeilen, wie eine LSTM-Schicht besitzt und so vielen Spalten, wie die Eingabe lang ist, ergibt. Diese Matrix muss transformiert werden (*permute*), damit sie mit den *hidden states* nach dem *Dropout* multipliziert (*Multiple_RNN_Weights*) werden kann. Die Matrix hat jetzt die Dimensionen [Eingabelänge, Anzahl Neuronen pro Schicht]. In einer Lambda-Schicht (*Context_Vector*) werden die Summen

der Gewichte spaltenweise gebildet, sodass sich ein Vektor der Länge 256 ergibt. Die Ausgabe besteht wie die Eingabe aus zwei Komponenten: der Tonhöhe (*note_height*) und der Tondauer (*duration*).

2 Training

Es wurden unterschiedliche Anzahlen an Epochen im Training getestet. Dabei wurde mit fünf Epochen begonnen und immer um fünf Epochen erhöht. Die maximale Anzahl an Epochen betrug 180. Wie die Validierung in Abschnitt Statistische Analyse zeigt, war das *Overfitting* bei einer Epochenzahl zwischen 70 und 80 noch gering. Für *early stopping* wurde als Parameter *patience* = 5 gesetzt, jedoch hatte das Einstellen der maximalen Epochenanzahl einen größeren Effekt als *early stopping* mit *patience* = 2. Als Gradientenverfahren wurde der *Mini-Batch-Gradient* genutzt. Um die optimale Größe der *Batch-size* zu determinieren, wurden verschiedene Größen ausprobiert und miteinander verglichen. Da die Ergebnisse keine sonderlich Große Varianz vorwies, wurde die Größe auf 32 gesetzt. Da dem Netz über die Eingabe aufgezeigt wird, welche Note als nächstes folgt, handelt es sich um überwachtes Lernen. Zudem wurden im Training drei unterschiedliche Trainingsdatensätze genutzt, sodass sich zum Schluss drei Modelle ergaben, die sich nur über die Trainingsdaten unterscheiden (siehe Abschnitt Normalisieren).

3 Hyperparameter

Neben den genannten *Hyperparametern* *Batch-size*, *Gradienten-verfahren*, *Dropouts*, *Neuronen-Anzahl*, *Early-Stop* und Epochenanzahl gibt es noch ein paar weitere Parameter die nun kurz erklärt werden. Als *Optimizer* wurde *Adam* genutzt, da dieser als Kombination von *SGD* und *RMSprop* viele Vorteile bietet und in der Literatur für den Einstieg oft empfohlen wird. Wei-

tere *Hyperparameter* finden sich in der MIDI-Generierung, da dort mit einer simulierten Wahrscheinlichkeitsverteilung ein Grad an Zufälligkeit reguliert werden kann, um nicht nur deterministische Ergebnisse zu erlangen. Dabei beschreibt ein Wert nahe 0 ein Ergebnis mit wenig Varianz, während ein Resultat nahe der 1 deutlich zufälligeres Muster aufweist. Desweiteren kann auch die Sequenzlänge bedingt reguliert werden (*additional_notes*).

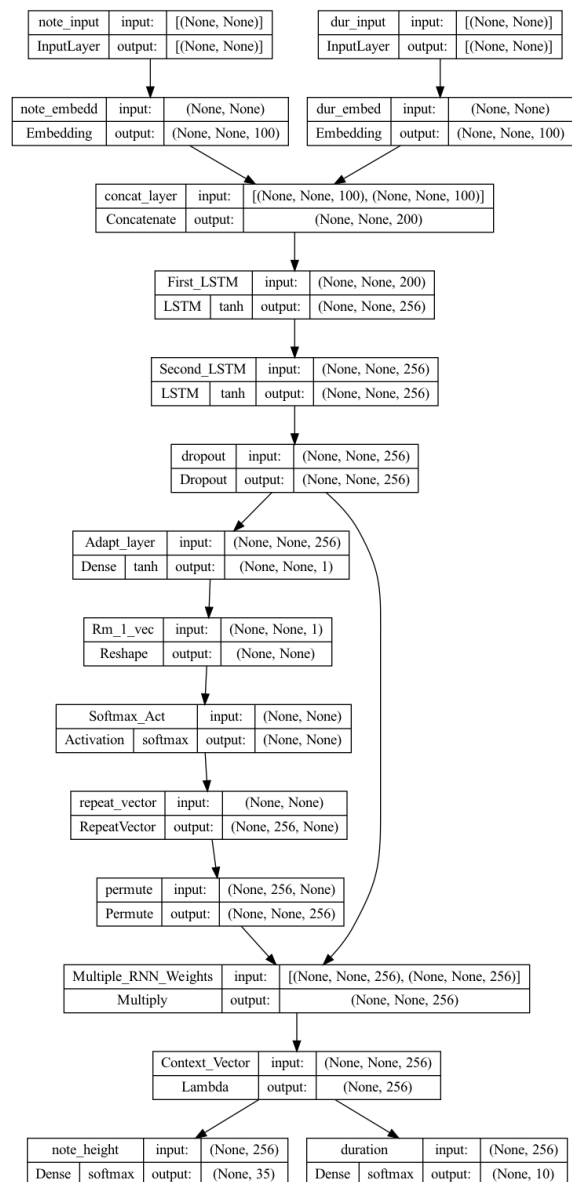


Abbildung 3. Architektur des Modells

4 MIDI-Generierung

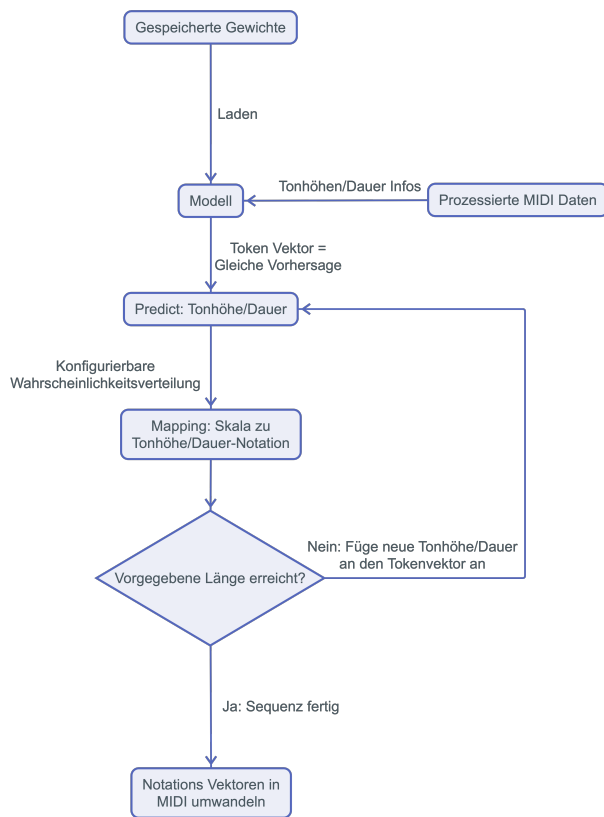


Abbildung 4. MIDI Generierung

Für die Generierung der *Licks* wurden die Gewichte des jeweiligen Modells und die notwendigen Informationen der vorprozessierten *Licks* geladen. Das Modell soll auf einer Eingabesequenz, die zu Beginn nur Platzhalter enthält, die nächste Note vorhersagen. Die Vorhersage des Modells wird jedoch nicht sofort an die Eingabesequenz angehängt, sondern als Grundlage für eine geringfügig zufällige Vorhersage genutzt. Die Methode `set_randomize_value` nimmt die Vorhersage des Modells und eine Zahl (`rand_val`) zwischen 0 und 1 entgegen. Dieser Zufallswert beträgt für die Tonhöhe 0,55 und für die Dauer 0,1. Ist der Wert 0, so wird die wahrscheinlichste Tonhöhe/Dauer ausgewählt. Sonst wird

anhand einer Wahrscheinlichkeitsverteilung, die auf `rand_val` und dem *Output* des Netzes basiert, die Note ausgewählt. Je näher der Wert von `rand_val` an 1 ist, um so zufälliger ist die Ausgabe. Die Rückgabe von `set_randomize_value` wird auf den entsprechenden Notenwert gesetzt und dieser der Eingabesequenz hinzugefügt. Die verlängerte Sequenz wird wieder dem Modell für eine Vorhersage übergeben. Dies wiederholt sich so lange, bis der *Lick* eine Länge von 17 (Default Parameter) erreicht hat. Die Vektoren der Tonhöhen und Tondauern werden dann verwendet, um daraus eine MIDI-Datei mithilfe des Pythonmoduls *music21* zu generieren. In Abbildung 4 ist dieser Prozess visualisiert. Die Gewichte und die *Lick*-informationen werden in das Modell geladen. Zur MIDI-Generierung wird ein *Token*-Vektor erstellt und daraufhin die nächste Tonhöhe/Tondauer von dem Modell vorhergesagt. Die Vorhersage wird wie bereits beschrieben für `set_randomize_value` als Grundlage genutzt und diese Ausgabe wird auf den entsprechenden Notenwert *gemappt*. Dies wird solange wiederholt, bis der *Token*-Vektor keine Platzhalter-*Token* mehr enthält. Der Vektor wird dann zur Generierung der fertigen MIDI-Datei genutzt. Die Menge an generierten *Licks* kann weiterhin in der Funktion `generate_n_licks` spezifiziert werden. Die neu generierten Licks befinden sich im entsprechenden Unterverzeichnis von `generated_midi`.

V Validierung

Da die generativen Daten sowohl objektive als auch subjektive Aspekte enthalten, wurde die Validierung in zwei Schritten durchgeführt. Zunächst wurde der objektive Anteil der Daten statistisch analysiert, danach wurde der subjektive Anteil in einer Variation des *Imitation Games* von Domänenexperten im Gebiet der Musik und teilweise auch speziell in der Jazz-Musik bewertet.

1 Statistische Analyse

Analysiert wurden die generierten Daten aus den Epochen 5, 35, 70, 80, 90 und 180 im Bezug auf die Verteilung der Tonhöhen, der Tondauern und das *Overfitting* auf die Trainingsdaten.

A Tonhöhe

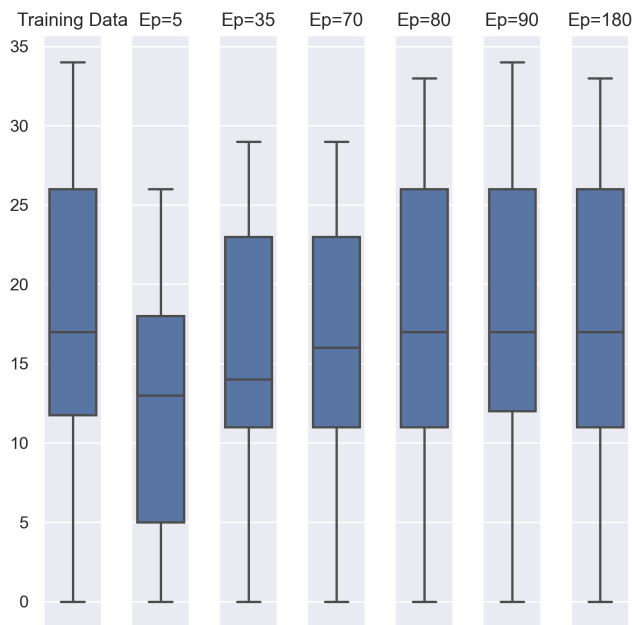


Abbildung 5. Darstellung der kodierten Tonhöhenverteilung nach Epoche

Abbildung 5 zeigt wie die Verteilung der Tonhöhen sich mit steigender Epoche den Trainingsdaten annähert. Durch das Beobachten des

Medians, lässt sich der Verallgemeinerungsprozess nachvollziehen. Epoche 5 weicht mit einem deutlich tieferen Median und einer deutlich geringeren Spannweite stark von den Trainingsdaten ab. Nach 70 Epochen liegt der Median annähernd auf der gleichen Höhe, wodurch die ersten 50% der Daten eine ähnliche Verteilung aufweisen. Mit höher werdender Epochenanzahl nähern sich die oberen 50% der Datenverteilung, an die Verteilung der Trainingsdaten an. Der Boxplot zeigt nur die relative Notenverteilung, nicht jedoch die absolute.

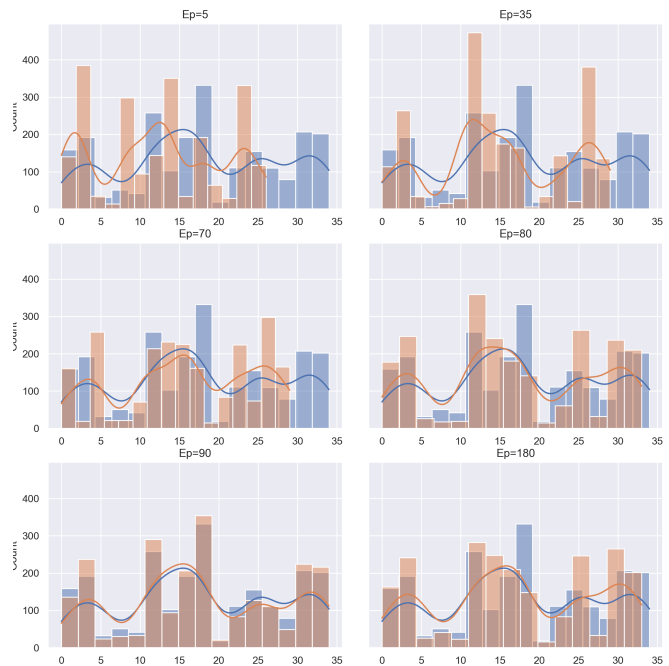


Abbildung 6. Histogramm: Tonhöhenverteilung nach Epoche und Note

Abbildung 6 gibt einen tieferen Einblick in die Verteilung, da neben den Epochen die Tonhöhen einzeln aufgetragen sind. Die Notenverteilung ist zu Beginn etwas gestaucht, das könnte darauf zurückzuführen sein, dass die Höhe der Kodierung vom ersten Vorkommen einer Note in den Trainingsdaten abhängt. Die selteneren Noten werden also nicht so häufig gelernt. Die Anzahl der Noten ist aber auch in den frühen Epochen bereits ähnlich, hier sieht man deutlich, dass sich erst ab Epoche 80 die Verteilung in den oberen

Rängen an die Trainingsdaten angleicht. Epoche 90 ist besonders erwähnenswert, da sie ein starkes *Overfitting* vermuten lässt, spätere Analysen zeigen aber, dass das nicht der Fall ist. Durch die Beobachtung des Histogramms, lässt sich der Verallgemeinerungsprozess des Netzes visuell nachvollziehen. Mit steigender Epochenanzahl, passt sich das Modell an die Tonhöhenverteilung zunehmend besser an. Dies zeigt sich vor allem durch die Dichtefunktion, die bei Epochen 80, 90, 180 nahezu gleich sind.

B Tondauer

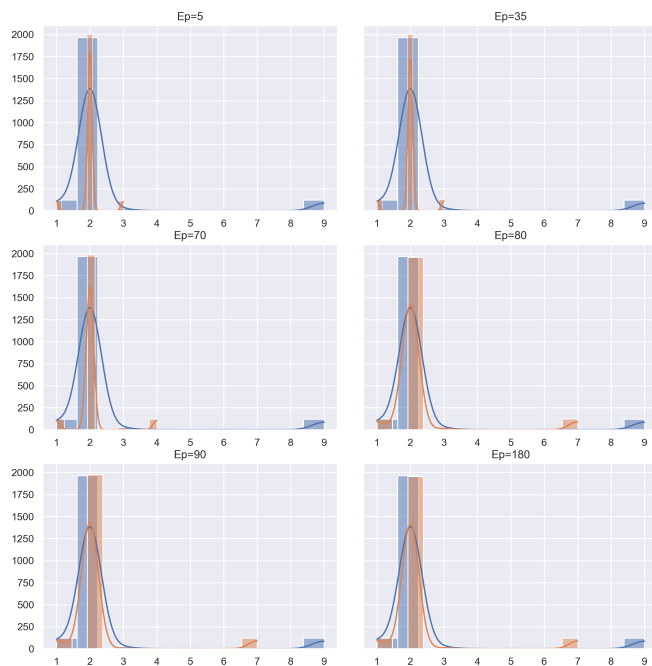


Abbildung 7. Darstellung der kodierten Tondauerverteilung nach Epoche

Abbildung 7 zeigt, dass die Tondauern größtenteils auf einen Wert beschränkt sind, daher sind auch die generierten Daten anfangs sehr konzentriert, passen sich jedoch langsam den Ausreißern an. Ab Epoche 80 verändert sich die Verteilung jedoch fast nicht mehr, die Extreme der Trainingsdaten werden nicht erreicht. Dies kann an den generierten *Licks*

auch eingesehen werden, da dort keine große Varianz an Rhythmischen-Elementen vorliegt (meistens Achtel-Ketten mit einer langen Notation am Ende). Dies wird vor allem durch eine der Normalisierungsmaßnahmen beschrieben, da rhythmische Anomalien geglättet wurden, um konsistentere Ergebnisse hinsichtlich der Anzahl an Trainingsdaten zu bekommen.

C Overfitting

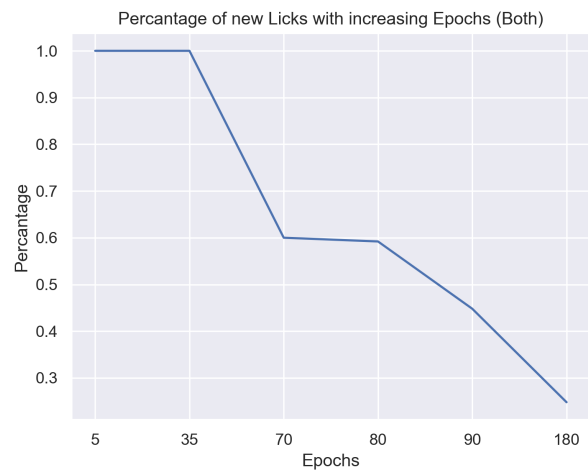


Abbildung 8. Prozentsatz der *Licks* die keine Kopie der Trainingsdaten darstellen

Abbildung 8 zeigt den Prozentsatz der generierten Daten, die keine exakte Kopie der Trainingsdaten darstellen. Der Prozentsatz kann ferner als relative Häufigkeit an neuen *Licks* beschrieben werden, da die Werte der Y-Achse mit der Anzahl an neuen *Licks* geteilt durch die Anzahl der generierten *Licks* berechnet wurde. In Abschnitt ?? wurde Epoche 90 als mögliches *overfitting* gewertet, hier ist zu sehen, dass zwar über 50% der Ergebnisse *overfitted* sind, jedoch noch über 40% neue Daten darstellen, die das Netz eigenständig erstellt hat. Somit zeigt sich das es einen *Sweet spot* in der Epochenanzahl gibt, da dort viele neue *Licks* vorliegen, die eine ähnliche Varianz bzgl. der Tonhöhenverteilung vorweisen.

2 Imitation Game

Die subjektiven Aspekte der generierten Daten wurden von Domänenexperten im Gebiet der Musik analysiert und ausgewertet. Dabei handelt es sich um Studierende und Dozenten von Musikinstitutionen in der Umgebung. In einer Variation des *Imitation Games* wurden aus den Trainings- sowie generativen Daten je 5 *Licks* zufällig ausgewählt die allen Experten blind vorgespielt wurden. Die Bewertung fand mit einem Bewertungsbogen statt, auf dem jeder *Lick* subjektiv mit einer Bewertung von 1 (schlecht) bis 10 (sehr gut) versehen werden, sowie eine Entscheidung, ob er computergeneriert ist, getroffen werden sollte. Zusätzlich wurde jeder Teilnehmer gefragt, ob er regelmäßig Jazz Musik hört bzw. praktiziert um die Ergebnisse in Expertengruppen zu differenzieren.

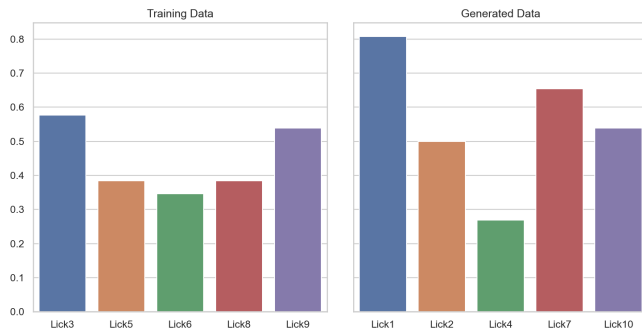


Abbildung 9. Bewertung der *Licks* nach "computergeneriert?" (0 - nein, 1 - ja)

Aus Abbildung 9 geht hervor, dass auch in den Trainingsdaten eine gewisse Unsicherheit herrscht, ob es sich nun um einen generierten *Lick* oder Trainingsdaten handelt. Die von der Mehrheit korrekt identifizierten *Licks* sind *Lick* 1, 7 und 10, aber auch *Lick* 3 und 9 wurden fälschlicherweise als generiert eingeschätzt. Die niedrigste Bewertung hat *Lick* 4 welcher von der KI generiert wurde, dieser wurde nur von knapp über 25% der Befragten als generiert eingeschätzt. Dadurch kann beobachtet werden, dass

die zufällig gezogenen generierten *Licks* große Unterschiede, hinsichtlich ihrer musikalischen Merkmale aufweisen. Durch Untersuchung der jeweiligen *Licks*, können kausale Ursachen für die Beobachtung gefunden werden, da beispielsweise *Lick* 1 eine Tonwiederholung vorweist, die sehr unnatürlich klingt. Entsprechend zeigt *Lick* 7 starke melodische Schwankungen hinsichtlich des Tonmaterials im 2. Takt, da dort teilweise alteriertes Tonmaterial genutzt wird, das sich nicht mit der harmonischen Begleitung deckt. Die daraus entstehende Dissonanz könnte die erhöhte Tendenz zu einem generierten *Lick* bestätigen. Auf der anderen Seite zeigt sich bei der Klassifizierung der *echten Licks* nur eine kleine Varianz und streut um Ca. 40% bzgl. einer Einstufung für generierte *Licks*.

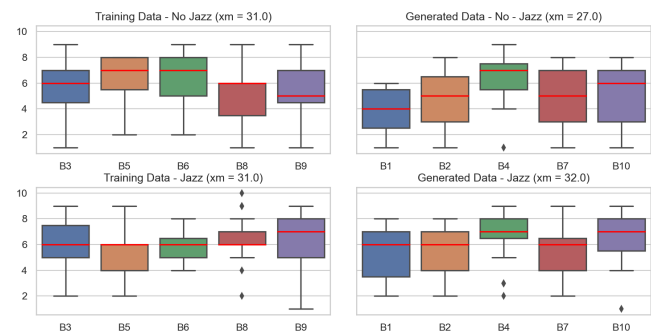


Abbildung 10. Bewertung der *Licks* nach persönlichem Empfinden (0 - schlecht, 10 - sehr gut)

In Abbildung 10 sind die subjektiven Bewertungen dargestellt, diese wurden in zwei Gruppen unterteilt, die obere Gruppe beinhaltet Domänenexperten in der Musik, nicht jedoch im Gebiet Jazz, die untere Gruppe beinhaltet Experten speziell im Gebiet Jazz. Klassische Musiker bewerten die Trainingsdaten teilweise ganz anders als die Jazz-Experten, vor allem in *Lick* B5 und B8 weichen die Bewertungen stark voneinander ab. Bei der Bewertung der generierten Daten ist die Übereinstimmung der Expertengruppen höher, generell bewerten die Experten im Gebiet Jazz die *Licks* aber höher. Wurde ein *Lick* als



generiert eingeschätzt ist seine Bewertung nicht zwingend schlecht, auch sehr eindeutige *Licks* wie B1 haben generell noch eine gute Bewertung. *Lick* B4 wurde allgemein am höchsten bewertet, mit nur zwei Ausreißern unter der 4 Punkte Marke. Um aus den Beobachtungen ein Maß zum Messen des subjektiven Eindrucks zu generieren, wurde der jeweilige Median einer Bewertungsverteilung eines *Licks* genommen und mit den anderen *Licks* der gleichen Kategorie (Training, Generiert und Jazz, Klassisch) summiert. Dadurch lassen sich die Beobachtungen auf ein Skalar reduzieren, das weiterhin untereinander verglichen werden kann. Dabei zeigt sich, dass die Gruppe der Jazz-Musiker die generierten *Licks*, mit einem summierten Median von 32, tendenziell am besten bewertet hat. Danach gleichen sich die summierten Mediane der beiden Musiker-Gruppen (31), während die generierten Daten von den klassischen Musikern am schlechtesten bewertet wurden.

Literatur

- [1] A. Spezzatti, “Neural networks for music generation.” <https://towardsdatascience.com/neural-networks-for-music-generation-97c983b50204>, 25.06.2019. [Online; letzter Zugriff: 07.01.2023].
 - [2] B. Jordan, “Keras-lstm-music-generator.” <https://github.com/jordan-bird/Keras-LSTM-Music-Generator>, 01.08.2021. [Online; letzter Zugriff: 07.01.2023].
 - [3] Skuldur, “Classical-piano-composer.” <https://github.com/Skuldur/Classical-Piano-Composer>, 29.12.2019. [Online; letzter Zugriff: 07.01.2023].
 - [4] Y. Verma, “A beginner’s guide to using attention layer in neural networks.” <https://analyticsindiamag.com/a-beginners-guide-to-using-attention-layer-in-neural-networks/>, 04.12.2021. [Online; letzter Zugriff: 07.01.2023].
 - [5] A. Géron, *Praxiseinstieg Machine Learning mit Scikit-Learn, Keras und TensorFlow: Konzepte, Tools und Techniken für intelligente Systeme. Aktuell zu TensorFlow 2*. O’Reilly, 2020.
 - [6] D. Bahdanau, K. Cho, and Y. Bengio, “Neural machine translation by jointly learning to align and translate,” *arXiv preprint arXiv:1409.0473*, 2014.
 - [7] uzaymacar, “attention-mechanism.” <https://github.com/uzaymacar/attention-mechanisms>, 26.12.2021. [Online; letzter Zugriff: 07.01.2023].
 - [8] R. Schwaiger and J. Steinwendner, *Neuronale Netze programmieren mit Python*. Rheinwerk Computing, 2019.
 - [9] D. Foster, *Generative deep learning: teaching machines to paint, write, compose, and play*. O’Reilly Media, 2019.
 - [10] A. A. Patel, *Hands-on unsupervised learning using Python: how to build applied machine learning solutions from unlabeled data*. O’Reilly Media, 2019.
 - [11] T. Rashid, *Neuronale Netze selbst programmieren: ein verständlicher Einstieg mit Python*. O’Reilly, 2017.
 - [12] F. Sikora, *Neue Jazz-Harmonielehre*. Schott Mainz, 2003.
-